CHALMERS
UNIVERSITY OF TECHNOLOGY | UNIVERSITY OF GOTHENBURG

A Chalmers University of Technology Master's thesis template for LaTeX

A Subtitle that can be Very Much Longer if Necessary

Master's thesis in Computer science and engineering

NAME FAMILYNAME

Department of Computer Science and Engineering
CHALMERS UNIVERSITY OF TECHNOLOGY
UNIVERSITY OF GOTHENBURG
Gothenburg, Sweden 2026

Master's thesis 2026

A Chalmers University of Technology Master's thesis template for LaTeX

A Subtitle that can be Very Much Longer if Necessary

NAME FAMILYNAME

UNIVERSITY OF GOTHENBURG

CHALMERS
UNIVERSITY OF TECHNOLOGY

Department of Computer Science and Engineering
Chalmers University of Technology
University of Gothenburg
Gothenburg, Sweden 2026

A Chalmers University of Technology Master's thesis template for LaTeX
A Subtitle that can be Very Much Longer if Necessary
NAME FAMILYNAME

© NAME FAMILYNAME, 2026.

Supervisor: Name, Department
Advisor: Name, Company or Institute (if applicable)
Examiner: Name, Department

Master's Thesis 2026
Department of Computer Science and Engineering
Chalmers University of Technology and University of Gothenburg
SE-412 96 Gothenburg
Telephone +46 31 772 1000

Cover: Description of the picture on the cover page (if applicable)

Typeset in LaTeX
Gothenburg, Sweden 2026

A Chalmers University of Technology Master's thesis template for LaTeX
A Subtitle that can be Very Much Longer if Necessary
NAME FAMILYNAME
Department of Computer Science and Engineering
Chalmers University of Technology and University of Gothenburg

Abstract

Abstract text about your project in Computer Science and Engineering.

Keywords: Computer, science, computer science, engineering, project, thesis.

Acknowledgements

Here, you can say thank you to your supervisor(s), company advisors and other people that supported you during your project.

Name Familyname, Gothenburg, 2026-03-05

Contents

List of Figures **xi**

List of Tables **xiii**

1 Introduction **1**
1.1 Aim 1
1.2 Problem 1
1.3 Limitations 2

2 Background **3**
2.1 The In-Vehicle Embedded System 3
2.2 In-Vehicle Networks and Event-Triggered Logging 3
2.3 Downstream Machine Learning Tasks and Data Quantity 3
2.4 Traditional Compression Methods 4
2.5 Rate-Utility trade-off and Releated Work 4

3 Methods **7**
3.1 Implementation Details 7
3.2 Experiment Setup 9

4 Results **11**

5 Conclusion **13**
5.1 Discussion 13
5.2 Conclusion 13

Bibliography **15**

A Appendix 1 **I**

List of Figures

List of Tables

1

Introduction

1.1 Aim

The aim of this project will be to investigate the use of neural compression techniques, specifically tokenization-based approaches, for compressing automotive time series data. The goal is to develop a compression framework that effectively balances the trade-off between compression ratio and utility for downstream machine learning (ML) tasks, such as predictive maintenance and anomaly detection.

The motivation behind using tokenization as a neural compression strategy for automotive time series data is to produce discrete latent representations that simplify the entropy modeling task within the compression pipeline. By constraining the data to a finite set of tokens, the complexity of modeling the underlying data distribution is reduced, enabling the use of lightweight entropy models that are computationally efficient. This is particularly advantageous in automotive applications where computational resources are limited, and real-time processing is often required.

The goal will be to design, implement and evaluate a tokenization-based neural compression framework tailored for time-series data. Evaluation will be conducted by comparing relevant parameters between the proposed framework and traditional compression methods.

1.2 Problem

The problem to be solved with this thesis is that of efficiently compressing vehicle telemetry data for downstream machine learning tasks. The issue is that vehicles generate a vast amount of telemetry data from various sensors and systems, which can be very challenging to store and transmit due to constraints on edge devices and communication bandwidth [1]. While techniques for data compression do exist, they are either not optimized for the specific characteristics of vehicle telemetry data or do not take into account the requirements of downstream machine learning tasks.

Traditional compression techniques such as event-triggered logging and algorithmic time series compression methods have limitations when applied to vehicle telemetry data. Event-triggered logging can miss important information that does not cross predefined thresholds, leading to incomplete data for machine learning models [2]–[4]. Algorithmic compression methods often rely on manually chosen parameters and

may not effectively capture the complex patterns present in vehicle telemetry data, which can negatively impact the performance of downstream machine learning tasks.

More recent approaches such as utility-aware adaptive telemetry and task-aware compression have shown promise in addressing some of these challenges. However, these methods often require complex policy learning or adaptive algorithms that may not be feasible for real-time applications in vehicles [5]–[8].

One less explored but promising approach to compressing data which could be suitable for vehicle telemetry data is that of neural compression in combination with tokenization. A system which incorporates a lightweight neural encoder and a small tokenizer should in theory be able to produce a lossy low entropy representation of the data which can be efficiently stored and transmitted. The main challenge will be to ensure that the compressed representation retains as much relevant information as possible for downstream machine learning tasks while still maintaining a high compression ratio.

1.3 Limitations

Due to time constraints and the scope of this thesis, there will be several limitations to the work presented:

- The work will be limited to only testing a single head / type of loss for the neural compression model. While it would be interesting to explore multiple heads, this would be time consuming in testing when running all the model variations.
- The work will only implement the encoder and tokenizer part of the pipeline while forgoing the entropy coding. This is done due to the focus of the thesis being on the neural compression step and while entropy coding is important in a full compression pipeline, it is less relevant for the research question at hand.
- The work will also forgo testing on real hardware as that would be very time consuming and difficult to set up while contributing relatively little to the main research question.
- The work will be limited to only testing regression tasks and anomaly detection as downstream tasks due to time constraints and structure of the available datasets. These two tasks should however be a good representation of common downstream tasks and adding more tasks would likely not drastically change the conclusion of the thesis.
- Due to wanting to keep the compressed data representation as versatile as possible, no task-aware compression techniques will be implemented. While task-aware compression could potentially yield better performance for specific tasks, it would limit the generalizability of the compressed representation and add significant complexity to the work. It also results in a more relevant comparison to other compression techniques which are not task-aware.

2

Background

2.1 The In-Vehicle Embedded System

An in-vehicle embedded system is a specialized computer system integrated within a vehicle to perform dedicated functions, often in real-time, and is essential for controlling, monitoring, and enhancing various automotive operations. These systems typically consist of both hardware and software components, such as electronic control units (ECUs), sensors, actuators, and communication interfaces, which are responsible for tasks like engine management, safety features, infotainment, and advanced driver assistance systems [9], [10].

2.2 In-Vehicle Networks and Event-Triggered Logging

Modern vehicles may contain dozens or even hundreds of these embedded systems, interconnected through in-vehicle networks (e.g., CAN, LIN, FlexRay, Ethernet), enabling efficient communication and coordination among different vehicle subsystems [9]–[11]. Event-triggered logging and diagnostic frameworks, which record data only when anomalies or threshold crossings occur, are often adopted to reduce data transmission and avoid bus saturation in complex systems, such as the in-vehicle embedded system. However, this selective approach can reduce holistic visibility of system health, as it may miss subtle degradation patterns or early warning signs that do not cross predefined thresholds. This complicates the detection of emerging faults and comprehensive condition monitoring [2]–[4]. Additionally, the need to carefully tune event thresholds and diagnostic criteria introduces maintenance challenges, as improper settings can lead to missed events or excessive false positives, further complicating system upkeep and reliability [2], [4].

2.3 Downstream Machine Learning Tasks and Data Quantity

Two developments in recent years further underline the shortcomings of event-triggered logging in automotive systems: the massive increase in signal-based data in the in-vehicle network and the growing relevance of downstream ML tasks.

Recent industry and research reports indicate that the data quantity generated by ADAS (Advanced Driver Assistance Systems) sensors in vehicles is growing at an extremely rapid pace. According to a 2023 technical paper referencing McKinsey's 2021 automotive electronics report, by 2030, about 95% of new vehicles will be connected, up from around 50% today, and a single car can generate up to 1 terabyte (TB) of data per hour from its sensors [1], [12]. This explosive growth is driven by the increasing number and sophistication of sensors—such as cameras, radars, and lidars—required for advanced safety and autonomous driving features, with the complexity and volume of data presenting significant challenges for storage, processing, and transmission within embedded automotive systems [1].

Modern vehicles increasingly rely on data-driven intelligence to enhance safety, reliability, and efficiency. Beyond perception and control, downstream ML tasks — those leveraging collected vehicle and sensor data for offline analysis, optimization and predictive functions — have become central to automotive-system design. These tasks include predictive maintenance [13], anomaly and intrusion detection [14], and fleet-level analytics like fuel consumption or maintenance scheduling [15].

Recent reviews highlight that while event-triggered and anomaly-based data collection can optimize resource use, they often result in fragmented or incomplete datasets, making it harder to implement robust predictive maintenance strategies and limiting the effectiveness of ML models that rely on continuous, high-resolution data streams [2], [3]. Multi-model and hybrid approaches are being explored to address these limitations, but the trade-off between data reduction and diagnostic completeness remains a significant challenge in both industrial and automotive contexts [3], [4]. How modern research approaches this trade-off is discussed in more detail in Section **??**.

2.4 Traditional Compression Methods

Traditional compression methods, based on information theory principles, often fall short in automotive applications, especially as a precursor for downstream ML tasks. For video/image compression, traditional methods like JPEG or MP3 are optimized for human perception (e.g., visual quality) rather than ML tasks or efficient downstream data use [16]. For time series data, algorithmic approaches like CHIMP or Gorilla depend on manually chosen parameters like window size and are sensitive to data characteristics such as entropy and signal variability. This limits their effectiveness in capturing the nuances required for accurate ML model performance in automotive contexts [17]. These algorithmic approaches were investigated by Johnsson [17] in a previous Master's thesis project. This work builds upon this thesis by exploring an alternative approach to compressing vehicle telemetry data.

2.5 Rate-Utility trade-off and Releated Work

Constructing downstream ML models for automotive systems, or in fact Internet-of-Things (IoT) systems in general, is a constant trade-off between handling large quantities of data and maximizing model performance. Traditional compression

techniques can reduce data volume, but often at the cost of losing critical information necessary for accurate ML tasks such as predictive maintenance, anomaly detection, and fleet analytics. The impact of this trade-off is well-documented in the literature. Muniz-Cuza, Jensen, Brusokas, *et al.* [18], for example, study the impact of lossy compression techniques on time series forecasting tasks and observe a constant trade-off between compression ratio and forecasting accuracy.

Existing research approaches these challenges from three different angles: utility-aware adaptive telemetry, neural compression, and task-aware compression.

- First, utility-aware adaptive telemetry methods aim to employ policy learning methods to dynamically adjust telemetry parameters to reduce maintenance costs while preserving data utility for downstream tasks. Although this approach is still emerging, recent research has demonstrated promising results [19].
- Second, neural compression techniques learn data representations optimized for both compression efficiency and ML task performance. This research is heavily inspired by deep generative models like GANs, VAEs, and autoregressive models, but focuses on compressing the data, instead of generating realistic data samples [20]. Neural compression techniques extend the introduced lossy compression principles in two key ways. First, they offer an alternative to traditional distribution modeling by leveraging deep neural networks to learn complex data distributions directly from the data, capturing intricate patterns and dependencies that traditional statistical models may miss. Second, they substitute traditional approaches to transform coding and quantization with learned representations [20]. Studies as early as 2019 have shown that neural compression methods can outperform traditional compression techniques for image and video data, especially at low bitrates [6]. The same has been shown for time series data [5], [8].
- Lastly, task-aware compression techniques focus on optimizing compression algorithms to retain information that is most relevant for specific tasks [20]. This idea has shown promise in handling time-series data more efficiently in IoT systems. Azar, Makhoul, Couturier, *et al.* [21] and Sun, Karras, and Zhang [22] for example explore task-aware compression algorithms that adaptively prioritize data features based on their relevance to downstream tasks, demonstrating improved performance in resource-constrained environments.

3

Methods

Within this section, we will discuss the methodology employed in our project to achieve the outlined aim. Specifically, we will discuss how the architecture of the baseline established compression model and the proposed tokenization-based compression framework are designed and implemented. We additionally discuss the experiment setup we will use to compare these two approaches.

3.1 Implementation Details

Baseline Model: First, we want to outline the architecture of the established compression method we will use as a baseline. We will use the autoencoder approach introduced by [5] as an inspiration for our autoencoder baseline model. It is important to note, that the paper introduces a sophisticated framework consisting of two models, the TCN-RNN model and the TCN-ARNN model, as well as a model selection mechanism. For simplicity sake, we will only focus on one of these models, the TCN-RNN model. The TCN-RNN model described in the paper is a representative of state-of-the-art continuous-latent neural time-series compression methods. Compared to vanilla autoencoders, the models architecture is explicitly designed for long temporal dependencies and has demonstrated stronger rate-distortion performance. This allows us to evaluate our tokenization-based approach against a competitive neural compressor rather than a toy model. Another point to note about the TCN-RNN model, is its costly architectural design, namely deep temporal convolutional stacks with large receptive fields, recurrent (often sequential) decoding, and dense continuous latent representations. These components typically lead to increased FLOPs, higher activation memory, and longer inference latency compared to discrete, token-based compression methods.

On a high level, the TCN-RNN architecture consists of the following components:

- Input handling and preprocessing
- TCN encoder
- RNN bottleneck
- RNN decoder
- TCN decoder

For the input handling and preprocessing it is important that the time-series data is segmented into fixed-length windows. These windows are treated as inputs to the model. Overlapping windows can be used to increase the effective training data size and improve reconstruction quality at window boundaries. Additionally, each channel is normalized independently using statistics computed on the training set.

The TCN encoder consists of a stack of Temporal Convolutional Network (TCN) blocks with increasing dilation factors. Each TCN block contains: A 1D convolution operating along the temporal dimension, normalization (BatchNorm or LayerNorm), a nonlinear activation function (ReLU or LeakyReLU), optional dropout for regularization, and a residual connection to stabilize training. The outcome is a continuous latent representation whose size is controlled by the downsampling factor and channel width.

The encoder output is processed by a recurrent neural network, specifically a gated recurrent unit (GRU), the RNN bottleneck. The RNN captures longer-range temporal dependencies that are not easily modeled by convolutions alone. At this stage, we produce a sequence of hidden states that serves as the core compressed signal. The hidden state dimensionality and number of layers define the strength of the bottleneck. The compression is achieved implicitly by restricting temporal resolution and latent dimensionality rather than by discretization. Afterwards, the RNN decoder reconstructs a latent sequence from the bottleneck representation. The decoder mirrors the bottleneck RNN in depth and hidden size.

Finally, a mirrored TCN stack upsamples the latent sequence back to the original temporal resolution. Transposed convolutions or interpolation-based upsampling are used. A final linear projection maps features back to the original channel dimension. The overall training objective is the reconstruction loss (e.g., MSE or MAE) between input and reconstructed signal.

Tokenization-Based Compression Framework: Next, we outline the architecture of our proposed tokenization-based compression framework. The architecture is inspired by recent advances in neural compression for speech and audio data, namely the Wavtokenizer paper. The key idea is to replace continuous latent representations with discrete tokens drawn from a learned codebook.

We again want to first outline the high-level components of the architecture:

- Input handling and preprocessing
- Lightweight encoder
- Vector quantization / tokenization
- Decoder head (for reconstruction) or downstream head (for task-specific loss)

The first component is (and should be) identical to the TCN-RNN baseline model. This ensures that any performance differences can be attributed to the compression method rather than preprocessing variations.

The encoder layer functionally has the same goal s the TCN encoder used in the baseline model, but has a different architecture. The encoder is designed to be

lightweight, avoiding heavy temporal convolutions or recurrent layers. Instead, a shallow neural network (e.g., small CNN or MLP applied per timestep) maps the input signal to intermediate embeddings. The encoder implements the learned transform stage of a lossy compression pipeline. Its purpose is not to predict labels or reconstruct the input per se, but to reshape the data into a representation that can be efficiently discretized. Specifically, the encoder learns a mapping that removes redundancy and correlations present in the raw time series, concentrates important information into a low-dimensional, stable representation, and aligns the geometry of the representation space with the constraints of quantization. The objective is usually joined with the quantization and decoder stages.

In contrast to autoencoder-based methods, where the encoder is free to produce high-entropy continuous latents, the encoder here is explicitly shaped to produce quantization-friendly representations that support discrete tokenization and entropy coding.

The next component is the vector quantization / tokenization stage. Here, intermediate embeddings are discretized using a learned codebook. The codebook very simply looks like this: codebook = nn.Parameter(torch.randn(K, D)), with K = number of tokens and D = embedding dimension. Each embedding vector is replaced by the index of its nearest codebook entry. This step introduces the only explicit source of information loss. The output is a sequence of discrete token IDs drawn from a finite vocabulary.

Lastly, we implement a head to enable loss calculation and backpropagation. Depending on the use case, this head can be a decoder for reconstruction tasks or a downstream head for task-specific losses (e.g., classification or regression). The head architecture is flexible and can be adapted to the specific application. The calculated loss is then used to jointly optimize the encoder, codebook, and head parameters.

3.2 Experiment Setup

To compare and evaluate the two methods outlined above, we will conduct a series of experiments. In this subsection we outline how we ensure a fair comparison, what metrics we will use, and what data we will use for the experiments.

Comparison and Metrics: There are three dimensions along which we want to compare the two compression methods. First, we want to evaluate the *compression performance* of both methods. This includes the metrics compression rate and reconstruction error. These metrics align with the evaluation standards in the neural compression literature. We will measure these metrics by training the two models on a reconstruction task and evaluating the reconstruction quality and effective compression rate. Second, we want to evaluate the *downstream task performance* when using compressed representations from both methods. This includes metrics such as accuracy for classification tasks or mean squared error for regression tasks. This evaluation reflects the practical utility of the compressed data for real-world applications. Third, we want to compare the *model efficiency* of both methods. This includes metrics such as parameter count, computational cost (FLOPs), inference la-

tency, and memory footprint. These metrics are important for assessing the feasibility of deploying the compression methods in resource-constrained environments.

An important note to make regarding comparability of these two metrics, especially regarding the downstream task performance, is that the two methods produce different types of compressed representations. The TCN-RNN baseline produces a continuous latent representation and the tokenization-based method produces discrete tokens. To ensure a fair comparison, we will need to design a lightweight unobtrusive adapter for each representation type that maps them into a common format suitable for the downstream model. The adapters should have minimal capacity to avoid introducing bias. This way, we can isolate the effect of the compression method itself on downstream performance. The envisioned design for the adapters is as follows:

- The adapter for the continuous latent representation will take the sequence of real-valued vectors output by the TCN-RNN model, average them over time to produce a single vector, and then pass this vector through a linear layer to match the input size of the downstream model.
- The adapter for the token-based representation will take the sequence of token IDs output by the tokenization model, map each token ID to a small vector using an embedding lookup, average these embeddings over time to produce a single vector, and then pass this vector through a linear layer to match the input size of the downstream model.

Data: The goal is to use industry relevant data, while also ensuring that the methods are tested on publicly available data. For that reason, we will use three datasets: Two Volvo datasets, the Volvo test fleet dataset and the Volvo electric mobility (EMOB) dataset, and a third publicly available dataset called X-CANIDS [23]. Each of these datasets contains multivariate time-series data collected from vehicles, including sensor readings, telemetry data, and operational parameters. The test fleet dataset will be used to construct a downstream regression task focused on predicting a continuous target variable. The EMOB dataset will be used to construct a downstream classification task, namely anomaly detection in battery systems. The publicly available dataset will be used as an additional benchmark to validate the generalizability of the findings across different data sources.

Downstream Models: Comparing the compression methods based on downstream task performanc requires downstream models. These models need to be simple enough to let us easily evaluate the compression methods used to compress the data which is fed into these models, but also still relevant to actual use cases. For this reason we will use four different downstream models to test the capabailities of the described compression methods. First, we will use the Volvo datasets to creat a regression (fuel consumption) and a classification (anomaly detection) downstream model. Second, we will use the X-CANIDS dataset to again create two models, one regression and one classification.

4

Results

Describe you results. Use tables, diagrams etc. for illustration.

5

Conclusion

You may consider to instead divide this chapter into discussion of the results and a summary.

5.1 Discussion

5.2 Conclusion

Bibliography

[1] R. Samantaray, "Adas sensor data handling in the world of autonomous mobility," *SAE Technical Paper Series*, 2023. DOI: 10.4271/2023-01-0993.

[2] P. Nunes, J. Santos, and E. Rocha, "Challenges in predictive maintenance – a review," *CIRP Journal of Manufacturing Science and Technology*, 2023. DOI: 10.1016/j.cirpj.2022.11.004.

[3] J. J. Montero Jiménez, S. Schwartz, R. Vingerhoeds, B. Grabot, and M. Salaün, "Towards multi-model approaches to predictive maintenance: A systematic literature survey on diagnostics and prognostics," *Journal of Manufacturing Systems*, 2020. DOI: 10.1016/j.jmsy.2020.07.008.

[4] K. Azar, Z. Hajiakhondi-Meybodi, and F. Naderkhani, "Semi-supervised clustering-based method for fault diagnosis and prognosis: A case study," *Reliability Engineering & System Safety*, vol. 222, p. 108 405, 2022. DOI: 10.1016/j.ress.2022.108405.

[5] Z. Zheng and Z. Zhang, "A temporal convolutional recurrent autoencoder based framework for compressing time series data," *Applied Soft Computing*, vol. 147, p. 110 797, 2023, ISSN: 1568-4946. DOI: https://doi.org/10.1016/j.asoc.2023.110797. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1568494623008153.

[6] J. Löhdefink, A. Bär, N. M. Schmidt, F. Hüger, P. Schlicht, and T. Fingscheidt, "Gan- vs. jpeg2000 image compression for distributed automotive perception: Higher peak snr does not mean better semantic segmentation," *arXiv preprint arXiv:1902.04311*, 2019. DOI: arXiv:1902.04311v1.

[7] M. Kawawa-Beaudan, R. Roggenkemper, and A. Zakhor, "Recognition-aware learned image compression," *Electronic Imaging*, vol. 34, no. 14, pp. 220-1-220–5, Jan. 2022, ISSN: 2470-1173. DOI: 10.2352/ei.2022.34.14.coimg-220. [Online]. Available: http://dx.doi.org/10.2352/EI.2022.34.14.COIMG-220.

[8] J. Liu, P. Djukic, M. Kulhandjian, and B. Kantarci, *Deep dict: Deep learning-based lossy time series compressor for iot data*, 2024. arXiv: 2401.10396 [eess.SP]. [Online]. Available: https://arxiv.org/abs/2401.10396.

[9] N. Navet and F. Simonot-Lion, *Automotive Embedded Systems Handbook*. CRC Press, 2017. DOI: 10.1201/9780849380273.

[10] R. E. Fairley, "Automobile embedded real-time systems," in *Systems Engineering of Software-Enabled Systems*. Wiley-IEEE Press, 2019, pp. 377–389. DOI: 10.1002/9781119535041.app2.

[11] L. L. Bello, R. Mariani, S. Mubeen, and S. Saponara, "Recent advances and trends in on-board embedded and networked automotive systems," *IEEE Transactions on Industrial Informatics*, vol. 15, pp. 1038–1051, 2019. DOI: 10.1109/tii.2018.2879544.

[12] M. Bertoncello, C. Martens, T. Möller, and T. Schneiderbauer, "Unlocking the full life-cycle value from connected-car data," McKinsey & Company, McKinsey Center for Future Mobility, Tech. Rep., Feb. 2021, White paper. [Online]. Available: https://www.mckinsey.com/industries/automotive-and-assembly/our-insights/unlocking-the-full-life-cycle-value-from-connected-car-data.

[13] A. Theissler, J. Pérez-Velázquez, M. Kettelgerdes, and G. Elger, "Predictive maintenance enabled by machine learning: Use cases and challenges in the automotive industry," *Reliability Engineering & System Safety*, vol. 215, 2021, ISSN: 0951-8320. DOI: https://doi.org/10.1016/j.ress.2021.107864. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0951832021003835.

[14] Ö. Özdemir, M. T. İşyapar, P. Karagöz, K. W. Schmidt, D. Demir, and N. A. Karagöz, *A survey of anomaly detection in in-vehicle networks*, 2024. arXiv: 2409.07505 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2409.07505.

[15] F. Chen, H. Jia, and W. Zhou, "Vehicle maintenance demand prediction: A survey," *Applied Sciences*, vol. 15, no. 20, 2025, ISSN: 2076-3417. DOI: 10.3390/app152011095. [Online]. Available: https://www.mdpi.com/2076-3417/15/20/11095.

[16] S. Ma, X. Zhang, C. Jia, Z. Zhao, S. Wang, and S. Wang, "Image and video compression with neural networks: A review," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 30, pp. 1683–1698, 2019. DOI: 10.1109/tcsvt.2019.2910119.

[17] S. Johnsson, "Large scale efficient data readout for vehicle fleets," M.S. thesis, Chalmers University of Technology, 2025.

[18] C. E. Muniz-Cuza, S. K. Jensen, J. Brusokas, N. Ho, and T. B. Pedersen, "Evaluating the impact of error-bounded lossy compression on time series forecasting," English, in *Advances in Database Technology - EDBT*, ser. Advances in Database Technology - EDBT, OpenProceedings, Mar. 2024, pp. 650–663. DOI: 10.48786/edbt.2024.56.

[19] P. Zhang, H. Zhang, Y. Pi, Z. Cao, J. Wang, and J. Liao, "Adapint: A flexible and adaptive in-band network telemetry system based on deep reinforcement learning," *IEEE Transactions on Network and Service Management*, vol. 21, pp. 5505–5520, 2023. DOI: 10.1109/tnsm.2024.3427403.

[20] Y. Yang, S. Mandt, and L. Theis, "An introduction to neural data compression," *Found. Trends Comput. Graph. Vis.*, vol. 15, pp. 113–200, 2022. DOI: 10.1561/0600000107.

[21] J. Azar, A. Makhoul, R. Couturier, and J. Demerjian, "Robust iot time series classification with data compression and deep learning," *Neurocomputing*, vol. 398, Feb. 2020. DOI: 10.1016/j.neucom.2020.02.097.

[22] G. Sun, P. Karras, and Q. Zhang, *Highly efficient direct analytics on semantic-aware time series data compression*, 2025. arXiv: 2503.13246 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2503.13246.

[23] S. Jeong, S. Lee, H. Lee, and H. K. Kim, "X-canids: Signal-aware explainable intrusion detection system for controller area network-based in-vehicle network," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 3, pp. 3230–3246, 2024. DOI: 10.1109/TVT.2023.3327275.

A

Appendix 1